PLUGIN DLL ARCHITECTURE .deckxclock / .dxclock

DeckX Clock & Weather Face Developer Guide

Single-Project C# DLL Plugin & Standby Face Architecture Specification

Interface: `IDeckXClockFace`

Target Hardware: DeckX DX8K (800×480 RGB565 / LVGL)

Platform: DeckX Control Studio v2.2.0+

1. Single-Project DLL Architecture Overview

Just like DeckX Action Plugins (`IDeckXPlugin`), custom **Standby Clock & Weather Faces** are developed as a **single C# project** that outputs a standard `.dll` assembly.

Developers do not need to manage multiple projects, external executables, or manual zip scripts. You simply implement the `IDeckXClockFace` interface, create your Settings UI inside the same project, and when you build or run the project once, DeckX's integrated MSBuild target:

> **The 1-Click Workflow (Hit 'Build' in Visual Studio or CLI):**
>
> 1. Compiles your C# code into a high-performance `.dll` assembly.
> 2. Bundles `clock.json` , `preview.png` , `settings.json` , and your DLL into `SampleClockFace.deckxclock` .
> 3. **Auto-Deploys** directly into `%APPDATA%\DeckX\devices\control_pad\dx8k\screensaver\clock_faces\<id>\` for instant hot-testing in DeckX Studio!

2. The `IDeckXClockFace` Interface

Defined in namespace `DeckX.Core.Plugins` , this is the core contract implemented by your clock face DLL:

```
using System;
using SkiaSharp;

namespace DeckX.Core.Plugins;

public interface IDeckXClockFace
{
    string Id { get; }            // e.g. "com.developer.neon_clock"
    string Name { get; }          // Display name on the gallery card
    string Author { get; }        // Developer / Studio name
    string Version { get; }       // Semantic version string (1.0.0)
    string Description { get; }   // Summary of features & typography

    // Return true to display the Settings (⚙) button on your card
    bool HasSettings => true;

    // Called when the user clicks the Settings (⚙) button in DeckX Studio
    void OpenSettings();

    // Lifecycle: called when loaded with the installed package path
    void OnInitialize(string packageDirectory) { }

    // Optional: custom SkiaSharp canvas preview rendering
    void RenderPreview(SKCanvas canvas, int width, int height, DateTime currentTime) { }

    // Lifecycle: called when deactivated or unloaded
    void OnShutdown() { }
}
```

3. Single Project Template Structure & NuGet Setup

Install the official `DeckX.Core` NuGet package via CLI:

```
dotnet add package DeckX.Core --version 1.0.0
dotnet add package SkiaSharp --version 2.88.9
```

Configure your `.csproj` file to reference `DeckX.Core` with `PrivateAssets="all"` and `ExcludeAssets="runtime"`:

```
<ItemGroup Condition="!Exists('..\..\DeckX.Core\DeckX.Core.csproj')">
  <PackageReference Include="DeckX.Core" Version="1.0.0">
    <PrivateAssets>all</PrivateAssets>
    <ExcludeAssets>runtime</ExcludeAssets>
  </PackageReference>
</ItemGroup>
```

All code, assets, and packaging configurations reside in a single directory:

```
templates/DeckX.ClockFace.SampleTemplate/
 ├── DeckX.ClockFace.SampleTemplate.csproj  <-- Single C# project with auto-pack target
 ├── SampleClockFace.cs                     <-- Implements IDeckXClockFace
 ├── SettingsDialog.cs                      <-- Integrated native dark settings UI
 ├── clock.json                             <-- Manifest (points to assembly DLL)
 ├── settings.json                          <-- User configuration schema
 ├── preview.png                            <-- 16:9 thumbnail for the card
 └── build-package.bat                      <-- Optional 1-click batch runner
```

`clock.json` Manifest Reference

The manifest file specifies the assembly DLL to load:

```
{
  "$schema": "https://deckx.app/schemas/clock-face.schema.json",
  "id": "com.deckx.samples.neonclock",
  "name": "Neon Modern Weather & Clock",
  "author": "DeckX Developer Community",
  "version": "1.0.0",
  "assembly": "DeckX.ClockFace.SampleTemplate.dll",
  "preview": "preview.png",
  "settings": true,
  "layout": "custom",
  "targetDevice": "DX8K",
  "resolution": {
    "width": 800,
    "height": 480
  }
}
```

Manifest Fields Matrix

Field	Type	Description
`id`	string	Unique package identifier (e.g. `com.company.face_name`).
`name`	string	Display name shown on the DeckX Studio Screen Saver card.
`assembly`	string	Filename of your compiled DLL (e.g. `MyClockFace.dll`).
`settings`	boolean	`true` shows the Settings (⚙) button; `false` hides it.
`preview`	string	Relative path to card thumbnail image (320×180 px recommended).
`resolution`	object	Hardware target canvas dimensions (`{"width": 800, "height": 480}`).

4. How the Settings Window Works

When the user clicks the **Settings (⚙)** button on your clock face card in DeckX Studio, DeckX checks the loaded plugin instance and directly invokes `clockFace.OpenSettings()`:

```
User clicks [Settings ⚙] in DeckX Studio │ ▼ DeckX locates loaded IDeckXClockFace instance matching target.Id │ ▼ DeckX calls: clockFace.OpenSettings() │ ▼ Your DLL displays its integrated SettingsDialog: ├── Reads settings.json from package directory ├── Displays Dark-themed GUI (City, API Key, °C/°F, 12h/24h, Colors) └── On "Save & Apply": Writes updated settings.json and closes cleanly!
```

Example Implementation (`SampleClockFace.cs`)

```
using System.Threading;
using DeckX.Core.Plugins;

namespace DeckX.ClockFace.SampleTemplate;

public class SampleClockFace : IDeckXClockFace
{
    private string _packageDirectory = string.Empty;

    public string Id => "com.deckx.samples.neonclock";
    public string Name => "Neon Modern Weather & Clock";
    public string Author => "DeckX Community";
    public string Version => "1.0.0";
    public string Description => "High-contrast neon standby clock with integrated settings UI dialog.
    public bool HasSettings => true;

    public void OnInitialize(string packageDirectory)
    {
        _packageDirectory = packageDirectory;
    }

    public void OpenSettings()
    {
        // Display native settings dialog on STA thread
        var thread = new Thread(() => SettingsDialog.ShowSettings(_packageDirectory));
        thread.SetApartmentState(ApartmentState.STA);
        thread.IsBackground = true;
        thread.Start();
    }

    public void OnShutdown() { }
}
```

5. Hardware Specifications & Canvas Metrics

The **DeckX DX8K** hardware specifications for standby display:

Hardware Attribute	Specification	Design Tip
Display Resolution	**800 × 480 pixels**	5:3 aspect ratio landscape format.
Color Format	RGB565 (16-bit color)	Optimized little-endian color depth for high frame rates.
Controller	ESP32-S3 Dual-Core @ 240MHz	8MB Octal PSRAM + 16MB Flash memory.
Graphics Engine	LVGL v8.3 Hardware Accelerated	Full anti-aliasing and sub-pixel typography.
Safe Area Margins	16px on all borders	Prevents text clipping against display bezels.

6. Step-by-Step Developer Tutorial

Step 1: Open the Project in Visual Studio or VS Code

Open the template project at:

```
templates/DeckX.ClockFace.SampleTemplate/DeckX.ClockFace.SampleTemplate.csproj
```

Step 2: Customize Your Clock Face Metadata

In `SampleClockFace.cs` and `clock.json`, change the `Id`, `Name`, `Author`, and `Description` to match your custom design.

Step 3: Add Your Settings Fields

In `SettingsDialog.cs`, add any custom controls (weather locations, API keys, color themes, display toggles) that your clock face supports.

Step 4: Build Once!

Run `dotnet build -c Release` or double-click **`build-package.bat`**.

```
> dotnet build -c Release
=== [DeckX Auto-Packer] Packaging SampleClockFace.deckxclock DLL... ===
=== [DeckX Auto-Packer] SUCCESS: Created SampleClockFace.deckxclock & Deployed to DeckX Studio! ===
```

Step 5: Test in DeckX Studio

1. Open **DeckX Control Studio** → **Device Settings** → **Screen Saver**.
2. Your custom clock face will already be installed and visible!
3. Click **Apply** to test standby mode, and click **Settings (⚙)** to open your integrated settings window.

Distribution Ready:
The compiled `releases/SampleClockFace.deckxclock` package can be shared directly with any DeckX user or uploaded to the DeckX Marketplace.